

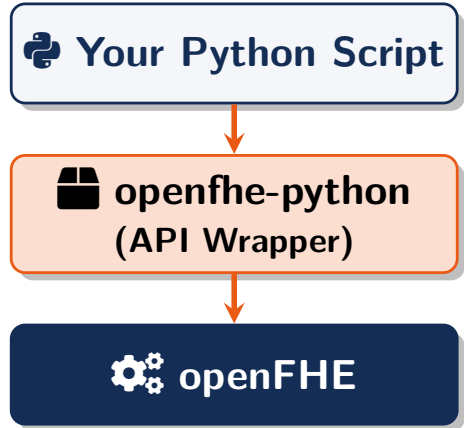
Fully Homomorphic Encryption (FHE)

Part III: Practical Introduction to OpenFHE (Python)

Ksawery Moźdzynski

What is OpenFHE?

- **OpenFHE** is a powerful, open-source FHE library written in C++.
- It provides a **unified API** for all major cryptographic FHE schemes.
- **Why openfhe-python?**
- It acts as a user-friendly wrapper. You write code in Python, but the computations run at the raw execution speed of the underlying C++ engine!



How to Install openfhe-python?



Linux & macOS

For most Unix-based systems, openfhe-python can be installed directly using the Python package manager. It is fast and requires no manual C++ compilation!

>_ Terminal

```
pip install openfhe
```



What about Windows?

Native compilation on Windows requires MSYS2 and MinGW64, which can be a complex and tedious process.

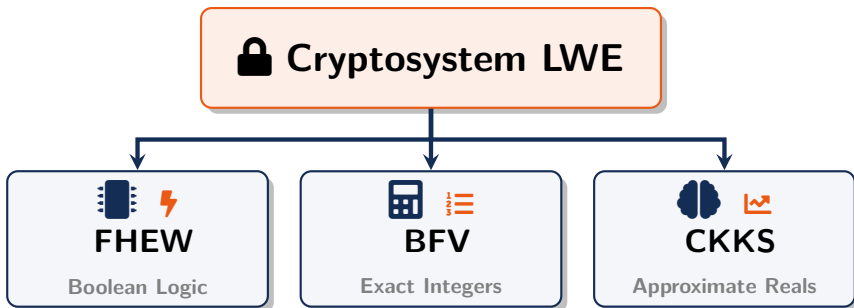
💡 The Recommended Way:

Use **WSL** (Windows Subsystem for Linux). Just open your WSL Ubuntu terminal and run the exact same `pip install` command as on Linux!

From Basic LWE to Advanced Schemes

💡 Your LWE knowledge is still valid!

The **LWE problem** from Day 2 is the foundation of modern FHE. Advanced algorithms are simply **highly optimized modifications** of LWE (e.g., using polynomials) to run faster!



1. FHEW Scheme: Boolean Logic



Fast Boolean Logic

- Optimized for evaluating Boolean logic gates (**AND, OR, XOR, NAND**).
- Processes data bit-by-bit rather than as large numbers.
- Features ultra-fast, per-gate **bootstrapping** (refreshing noise in milliseconds).
- **Best for:** Arbitrary logic circuits, non-linear functions (e.g., comparisons, if-else statements).

1. FHEW Scheme: Context & Parameters

Boolean FHE requires a specialized manager: `BinFHEContext`.

```
1  from openfhe import *
2
3  # 1. Setup Boolean Context and Parameters
4  # BinFHEContext is the primary object/manager for Boolean FHE.
5  bin_context = BinFHEContext()
6
7  # STD128: Configures standard 128-bit security level.
8  # GINX: Chooses the CGGI (TFHE) bootstrapping method, which is
9  #       currently the fastest for boolean logic.
10 # (AP method is alternatively used for the DM/FHEW scheme)
11 bin_context.GenerateBinFHEContext(BINFHE_PARAMSET.STD128, BINFHE_METHOD.GINX)
```

1. FHEW Scheme: Key Generation

Boolean schemes require a private key (for you) and massive **Bootstrapping Keys** (for the cloud).

```
14  # 2. Key Generation
15  # Generate the standard LWE secret key (sk).
16  # Keep this safe! It is used to encrypt and decrypt your data.
17  sk = bin_context.KeyGen()
18
19  # Generate Bootstrapping Keys (BTKey).
20  # 1. This is a public evaluation key that you send to the cloud.
21  # 2. The cloud uses it to blindly "wash away" noise after EVERY logic gate!
22  # 3. It is secure: third parties CANNOT derive your private key (sk)
23  #     from it, and CANNOT decrypt your data!
24  bin_context.BTKeyGen(sk)
```

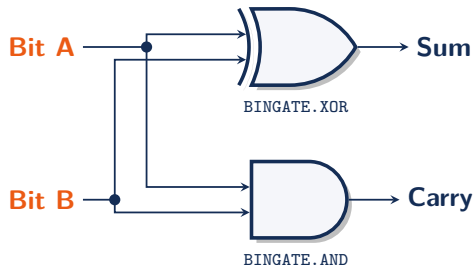

1. FHEW Scheme: Gate Evaluation

All Boolean logic operations are natively supported via the context.

```
32 # 4. Evaluate an AND gate homomorphically
33 # BINGATE.AND instructs the FHE engine to compute: ct1 AND ct2.
34 # Other supported gates include: OR, XOR, NAND, NOR, XNOR.
35 #
36 # CRUCIAL DETAIL:
37 # EvalBinGate evaluates the logic gate AND automatically calls
38 # the fast Bootstrapping subroutine to reduce noise!
39 ct_result = bin_context.EvalBinGate(BINGATE.AND, ct1, ct2)
```


Time for Practice: Building a Half-Adder

The Half-Adder Circuit



Task: Encrypted Half-Adder

Write a Python script using `openfhe-python` to implement the Half-Adder circuit on encrypted data!

1. **Setup:** Create `BinFHEContext` and generate secret & bootstrapping keys.
2. **Loop:** Iterate 4 inputs: (0,0) to (1,1).
3. **Encrypt** the inputs A and B.
4. **Evaluate** homomorphically:
 - `Sum = EvalBinGate(BINGATE.XOR,...)`
 - `Carry = EvalBinGate(BINGATE.AND,...)`
5. **Decrypt** the outputs and print results.

Solution: The FHE "IF-ELSE" Statement

Truth Table Analysis

C (Control)	Bit A	Bit B	Output
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

- When $C = 0$, Output = Bit A
- When $C = 1$, Output = Bit B

The Multiplexer (MUX)

You built a **MUX**, the FHE equivalent of conditional logic:

```
if (C == 1) return B; else return A;
```

Why is this crucial?

The cloud cannot read encrypted C , making standard if-else branching impossible!

Instead, it computes **both** branches and mathematically filters the result.

Native CMUX Gate

OpenFHE provides a native, optimized gate for this:

```
EvalBinGate(BINGATE.CMUX, [C, B, A])
```


BFV Setup (2/3): Enabling Features



```
8  # By default, OpenFHE disables all features to save memory.
9  # We must explicitly enable the cryptographic capabilities we need.
10
11 # Unlocks basic cryptographic operations: key generation,
12 # encryption (Encrypt), and decryption (Decrypt).
13 crypto_context.Enable(PKESchemeFeature.PKE)
14
15 # Enables Key Switching. This is a core mechanism absolutely
16 # necessary for vector multiplication and rotation in the cloud.
17 crypto_context.Enable(PKESchemeFeature.KEYSWITCH)
18
19 # Unlocks homomorphic mathematical operations on the server,
20 # giving us access to methods like EvalAdd and EvalMult.
21 crypto_context.Enable(PKESchemeFeature.LEVELEDSHE)
```

BFV Setup (3/3): Key Generation



```
13 # =====
14 # Key Generation
15 # =====
16
17 # KeyGen() returns a KeyPair object containing the standard
18 # private key (for decryption) and public key (for encryption).
19 key_pair = crypto_context.KeyGen()
20
21 # Multiplying ciphertexts makes them grow too large to handle.
22 # EvalMultKeyGen() creates a special "Multiplication Key".
23 # It allows the cloud server to mathematically "compress" the
24 # result back to its normal size after each multiplication.
25 # This keeps computations fast, without exposing your secret key!
26 crypto_context.EvalMultKeyGen(key_pair.secretKey)
```

BFV: SIMD Encoding & Encrypt (Client)

```
18  # =====
19  # SIMD Encoding & Encryption
20  # =====
21  vec_a = [ 2, 4, 6, 8 ]
22  vec_b = [ 3, 2, 1, 5 ]
23
24  # Packs the integer arrays into single Plaintext objects.
25  # This enables SIMD Batching: packing multiple numbers into one
26  # structure to perform parallel FHE operations on all of them.
27  ptxt_a = crypto_context.MakePackedPlaintext(vec_a)
28  ptxt_b = crypto_context.MakePackedPlaintext(vec_b)
29
30  # Encrypts the packed plaintexts using the Public Key.
31  # The resulting ciphertexts can now be safely sent to the cloud.
32  ctxt_a = crypto_context.Encrypt(key_pair.publicKey, ptxt_a)
33  ctxt_b = crypto_context.Encrypt(key_pair.publicKey, ptxt_b)
```

BFV: Evaluation (Cloud)



```
29 # =====
30 # Homomorphic Evaluation (Cloud)
31 # =====
32 # The ciphertexts are sent to the untrusted cloud.
33 # The server computes on them without ever knowing the
34 # underlying plaintext data or the secret key.
35
36 print("Evaluating EvalMult on encrypted vectors...")
37
38 # EvalMult multiplies ALL corresponding elements in the
39 # packed arrays simultaneously (SIMD Batching).
40 # It automatically uses the Multiplication Key stored in
41 # the CryptoContext to mathematically "compress" (relinearize)
42 # the resulting ciphertext back to its normal size.
43 ctxt_result = crypto_context.EvalMult(ctxt_a, ctxt_b)
```

BFV: Decrypt & Extract (Client)

```
38 # =====
39 # Decrypt & Extract (Client)
40 # =====
41 # The cloud returns the evaluated ciphertext. The client
42 # decrypts it using their secret key to recover the Plaintext.
43
44 result_ptxt = crypto_context.Decrypt(key_pair.secretKey, ctxt_result)
45
46 # GetPackedValue() extracts the underlying vector from the
47 # Plaintext object. We slice [:4] to see only our actual
48 # data, ignoring the padding zeros added by the ring dimension.
49
50 print(f"Decrypted Result: {result_ptxt.GetPackedValue()[:4]}")
51
52 # Expected Console Output:
53 # Decrypted Result: [ 6,  8,  6, 40 ]
```

BGV: The Missing Bootstrapping (1/2)

Why doesn't BGV have an EvalBootstrap() method?

BGV is an exact integer scheme. While a bootstrapping algorithm theoretically exists for it (based on *digit extraction*), it has a **terrifying computational complexity**. Because of this extreme overhead, OpenFHE does NOT expose it in its standard API.

⚠ The Danger of Deep Circuits

Without bootstrapping, we cannot "refresh" the noise on the fly mid-computation. If we multiply ciphertexts too many times, the noise explosion will irreversibly destroy the data!

BGV: The Missing Bootstrapping (2/2)

How do we compute without Bootstrapping?

Since we cannot reset the cryptographic noise in the cloud, we must plan our entire circuit's depth before the encryption even begins.

✓ The OpenFHE Solution: Pre-allocation

Instead of relying on bootstrapping, developers must define a strict noise budget upfront using the `MultiplicativeDepth` parameter. This dictates exactly how many sequential multiplications the cloud can safely perform before the data is lost.

BGV in Practice: Multiplicative Depth (1/2)



```
1 def run_bgv_scenario(depth: int, num_multiplications: int):
2     # 1. Setup: Define the maximum sequential multiplications
3     parameters = CCParamsBGVRNS()
4     parameters.SetMultiplicativeDepth(depth)
5
6     # ... (GenCryptoContext, KeyGen, Encrypt vector) ...
7
8     current_ctxt = ctxt # <-- it's an encrypted vector [1,2,3,4]
9
10    # 2. Evaluation: Each EvalMult consumes the depth budget!
11    for i in range(num_multiplications):
12        current_ctxt = crypto_context.EvalMult(current_ctxt, ctxt)
13
14    # 3. Decryption
15    result = crypto_context.Decrypt(key_pair.secretKey, current_ctxt)
16    print(f"Decrypted: {result.GetPackedValue()[:4]}\n")
```

BGV in Practice: Multiplicative Depth (2/2)

```
24 if __name__ == "__main__":
25     # -----
26     # Scenario 1: Budget exceeded
27     # -----
28     # We allocate a budget for 1 multiplication, but perform 2.
29     # The noise explosion will irrevocably corrupt the data (garbage)
30     print("--- BGV test with MultiplicativeDepth = 1 ---")
31     run_bgv_scenario(depth=1, num_multiplications=2)
32
33     # -----
34     # Scenario 2: Budget adjusted (The OpenFHE Solution for BGV)
35     # -----
36     # Since OpenFHE does not support EvalBootstrap() for BGV,
37     # we must correctly allocate the noise budget in advance.
38     print("--- BGV test with MultiplicativeDepth = 2 ---")
39     run_bgv_scenario(depth=2, num_multiplications=2)
```

BGV in Practice: Console Output

! Scenario 1: Budget Exceeded (Depth = 1)

```
--- BGV test with MultiplicativeDepth = 1 ---  
Original vector: [1, 2, 3, 4]  
Performing multiplication 1...  
Performing multiplication 2...  
Decrypted result: [-26843, -16257, 702, 2507]
```

BGV in Practice: Console Output

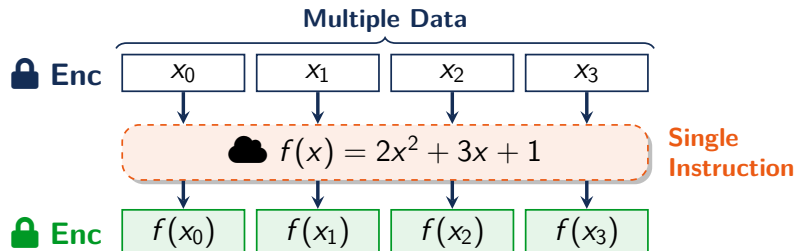
✓ Scenario 2: Budget Adjusted (Depth = 2)

```
--- BGV test with MultiplicativeDepth = 2 ---  
Original vector: [1, 2, 3, 4]  
Performing multiplication 1...  
Performing multiplication 2...  
Decrypted result: [1, 8, 27, 64]
```

Polynomial Evaluation Example

Computing $f(x) = 2x^2 + 3x + 1$ in Parallel

The **BGV** scheme encodes data in packed vectors. When we evaluate the polynomial homomorphically, the cloud uses **SIMD** (*Single Instruction, Multiple Data*) architecture. This means the exact same mathematical operations are applied to all encrypted slots simultaneously!



Evaluating $f(x) = 2x^2 + 3x + 1$ (BGV CryptoContext Configuration)

```
1 from openfhe import *
2
3 parameters = CCParamsBGVRNS()
4 parameters.SetPlaintextModulus(65537)
5
6 # We reserve a multiplicative depth of 2:
7 # - 1 level for EvalSquare ( $x^2$ )
8 # - 1 level for EvalMult (multiplying  $x^2$  by the constant 2)
9 parameters.SetMultiplicativeDepth(2)
10 crypto_context = GenCryptoContext(parameters)
```

Evaluating $f(x) = 2x^2 + 3x + 1$ (BGV CryptoContext Configuration)

```
11 # Enable features required for our operations:  
12 # PKE (Public Key Encryption): Enables basic cryptographic  
13 # functions like KeyGen(), Encrypt(), and Decrypt().  
14 crypto_context.Enable(PKESchemeFeature.PKE)  
15  
16 # KEYSWITCH: Enables key switching. This is crucial for  
17 # relinearization (flattening the ciphertext back to its  
18 # normal size after EvalSquare or EvalMult).  
19 crypto_context.Enable(PKESchemeFeature.KEYSWITCH)  
20  
21 # LEVELEDSHE: Enables mathematical operations on ciphertexts  
22 # (EvalAdd, EvalMult, EvalSquare) within the depth budget.  
23 crypto_context.Enable(PKESchemeFeature.LEVELEDSHE)
```

Evaluating $f(x) = 2x^2 + 3x + 1$ (Key Generation)

```
24 key_pair = crypto_context.KeyGen()  
25  
26 # EvalMultKeyGen generates evaluation keys (relinearization keys).  
27 # This is absolutely required before calling EvalSquare() or EvalMult(),  
28 # allowing the cloud to compute products securely.  
29 crypto_context.EvalMultKeyGen(key_pair.secretKey)  
30 key_pair = crypto_context.KeyGen()
```

Evaluating $f(x) = 2x^2 + 3x + 1$ (Data Encryption)

```
31  vec = [1, 2, 3, 4]
32
33  # Pack the vector into a Plaintext object to compute in SIMD mode
34  ptxt = crypto_context.MakePackedPlaintext(vec)
35  ctxt = crypto_context.Encrypt(key_pair.publicKey, ptxt)
```

Evaluating $f(x) = 2x^2 + 3x + 1$ (Encoding Constants as Plaintexts)

```
36 # In BGV (SIMD mode), we cannot multiply a ciphertext directly
37 # by a raw integer scalar. We must duplicate each constant across
38 # the entire length of the input vector and encode them as
39 # Plaintext objects so SIMD operations work on every slot.
40
41 vec_len = len(vec)
42
43 ptxt_2 = crypto_context.MakePackedPlaintext([2] * vec_len) # Creates [2, 2, 2, 2]
44 ptxt_3 = crypto_context.MakePackedPlaintext([3] * vec_len) # Creates [3, 3, 3, 3]
45 ptxt_1 = crypto_context.MakePackedPlaintext([1] * vec_len) # Creates [1, 1, 1, 1]
```

Evaluating $f(x) = 2x^2 + 3x + 1$ (Manual Polynomial Evaluation in BGV)

```
46 # STEP A:  $x^2$  (Consumes 1 level of multiplicative depth)  
47 ctxt_x2 = crypto_context.EvalSquare(ctxt)  
48  
49 # STEP B:  $2 * x^2$  (Consumes 1 level of multiplicative depth)  
50 ctxt_2x2 = crypto_context.EvalMult(ctxt_x2, ptxt_2)  
51  
52 # STEP C:  $3 * x$   
53 ctxt_3x = crypto_context.EvalMult(ctxt, ptxt_3)  
54  
55 # STEP D:  $(2x^2) + (3x)$  (EvalAdd adds very little noise)  
56 ctxt_res = crypto_context.EvalAdd(ctxt_2x2, ctxt_3x)  
57  
58 # STEP E: Adding the constant 1 at the end: + 1  
59 ctxt_res = crypto_context.EvalAdd(ctxt_res, ptxt_1)
```

Evaluating $f(x) = 2x^2 + 3x + 1$ (Decryption of the Results)

```
60 result_ptxt = crypto_context.Decrypt(key_pair.secretKey, ctxt_res)
61
62 # Extract only the valid elements (ignoring trailing zeros)
63 result_vec = result_ptxt.GetPackedValue()[ :vec_len]
64
65 print(f"Decrypted result: {result_vec}\n")
```

Evaluating $f(x) = 2x^2 + 3x + 1$ (Results)

>_ Console Output

```
Original vector: [0, 1, 2, 3]
Computing polynomial manually:  $f(x) = 2x^2 + 3x + 1$ 
Decrypted result: [1, 6, 15, 28]
```

✓ Math Verification (SIMD in action):

Notice how the cloud correctly evaluated the polynomial for all slots simultaneously:

$$f(0) = 2(0)^2 + 3(0) + 1 = 1$$

$$f(1) = 2(1)^2 + 3(1) + 1 = 6$$

$$f(2) = 2(2)^2 + 3(2) + 1 = 15$$

$$f(3) = 2(3)^2 + 3(3) + 1 = 28$$

Time for Practice: Secure Polynomial Evaluation (BGV)

Task: Cloud Risk Assessment using SIMD

Write a Python script using `openfhe-python` to evaluate the risk function $f(x) = x^4 + 2x^3 + 3x^2 + 4$ on a batch of patients $x = [0, 7, 12, 2]$ using the BGV scheme.

- 1. Calculate Depth:** Manually determine the exact `MultiplicativeDepth` required to evaluate the polynomial.
- 2. Setup:** Configure `CCParamsBGVRNS` with `PlaintextModulus=65537`. Enable necessary features (PKE, `KEYSWITCH`, `LEVELED SHE`) and generate the `EvalMultKey`.
- 3. SIMD Packing:** Encode the input vector and the constants (2, 3, and 4) as packed vectors across all slots using `MakePackedPlaintext`.
- 4. Evaluate & Verify:** Compute $f(x)$ homomorphically using `EvalSquare`, `EvalMult`, and `EvalAdd`. Decrypt and extract the result.

Time for Practice: Secure E-Voting System

The Voting Scenario

- **Voters:** Each voter casts a vote as a vector. They encrypt it using the **Public Key** (PKE).
- **Cloud Server:** Receives encrypted votes and homomorphically sums them without decrypting or knowing the private key.
- **Election Committee:** Uses the **Secret Key** to decrypt the final sum.



Task: Encrypted Summing in BGV

Write a Python script using `openfhe-python` to implement the E-Voting system!

- **Setup:** Create `CCParamsBGVRNS`, set depth to 2, and enable PKE and `LEVELEDSE`.
- **KeyGen:** Generate a `KeyPair` for the election.
- **Encrypt:** Loop through 5 simulated vote vectors. Encode them using `MakePackedPlaintext` and encrypt using the **Public Key**.
- **Evaluate:** Sum the votes sequentially using `EvalAdd`.
- **Decrypt:** Decrypt the sum with the **Private Key**, restrict the length to 5 using `SetLength(5)`, and print the final sum.

Time for Practice: Secure Portfolio Valuation (BFV)

The Scenario

A client holds a portfolio of exactly 4 assets and wants the cloud server to compute its total value based on current market prices.

- **Client:** Encrypts the quantities of the assets (e.g., [1, 2, 0, 11]).
- **Server:** Holds the current market prices in plaintext (e.g., [10, 3, 4, 2]).
- **Goal:** Use SIMD batching and the homomorphic Dot Product operation to compute the final value securely.

Task: BFV Inner Product

Write a Python script using `openfhe-python` to implement this computation!

1. **Setup:** Configure `CCParamsBFVRNS`. Set `PlaintextModulus` to 65537, `MultiplicativeDepth` to 2, and crucially, `BatchSize` to 4.
2. **Enable Features:** Enable PKE, `KEYSWITCH`, `LEVELED SHE`, and `ADVANCED SHE` (required for aggregation).
3. **Keys:** Generate the secret/public keys, `EvalMultKey`, and `EvalSumKey`.
4. **Encode & Encrypt:** Pack both vectors using `MakePackedPlaintext`. Encrypt the quantities, keep prices as Plaintext.
5. **Evaluate & Decrypt:** Compute the portfolio value using `EvalInnerProduct`. Decrypt and extract the scalar result!

3. CKKS Scheme



Approximate Real-Number Arithmetic

- The only scheme that natively handles **floating-point / real numbers**.
- It is an *approximate* scheme (like floating-point math in regular CPUs, it introduces tiny rounding errors).
- **Best for:** Machine Learning (AI inferences), polynomial approximations, statistical modeling.

CKSS Example: Secure Cloud Risk Assessment

Encrypted AI Inference

- Scenario:** A cloud server evaluates a single artificial neuron equation: $y = \sum_{i=1}^n (x_i \cdot w_i) + b$.
- Encryption:** The client encrypts medical results x_i as a packed floating-point vector using `MakeCKKSPackedPlaintext`.
- Evaluation:** The cloud homomorphically multiplies the encrypted features by the plaintext weights w_i , sums the resulting vector using `EvalSum` (remember to generate the sum key!), and adds the scalar bias b .
- Decryption:** The client decrypts the vector using `GetRealPackedValue` and extracts its first position to get the final risk score.

CKKS in Practice: Setup



```
1 from openfhe import *
2
3 # 1. Setup CryptoContext for CKKS scheme
4 parameters = CCParamsCKKSRNS()
5 parameters.SetMultiplicativeDepth(1)
6 parameters.SetScalingModSize(50)
7 parameters.SetBatchSize(4)
8
9 crypto_context = GenCryptoContext(parameters)
```

CKKS in Practice: Features & Keys

```
11 # Enable required features (ADVANCEDSHE is needed for EvalSum)
12 crypto_context.Enable(PKESchemeFeature.PKE)
13 crypto_context.Enable(PKESchemeFeature.KEYSWITCH)
14 crypto_context.Enable(PKESchemeFeature.LEVELED SHE)
15 crypto_context.Enable(PKESchemeFeature.ADVANCEDSHE)
16
17 # 2. Key Generation
18 key_pair = crypto_context.KeyGen()
19 crypto_context.EvalMultKeyGen(key_pair.secretKey)
20 crypto_context.EvalSumKeyGen(key_pair.secretKey)
```

CKKS in Practice: Encrypt Data



```
21 # 3. Client Side: Encrypt patient medical data
22 patient_features = [1.2, 3.5, 0.8, 2.1]
23 ptxt_features = crypto_context.MakeCKKSPackedPlaintext(patient_features)
24 ctxt_features = crypto_context.Encrypt(key_pair.publicKey, ptxt_features)
25
26 # 4. Cloud Side: Cloud AI Model parameters
27 model_weights = [0.5, -1.2, 2.0, 0.9]
28 bias = 0.75
29 ptxt_weights = crypto_context.MakeCKKSPackedPlaintext(model_weights)
```

CKKS in Practice: Evaluate Neural Network



```
30 # Evaluate the neuron:  $y = \sum(x_i * w_i) + b$ 
31 # Step A: Element-wise multiplication of features and weights
32 ctxt_mult = crypto_context.EvalMult(ctxt_features, ptxt_weights)
33
34 # Step B: Sum all components in the packed ciphertext vector
35 ctxt_sum = crypto_context.EvalSum(ctxt_mult, 4)
36
37 # Step C: Add the bias (scalar addition natively supported in CKKS)
38 ctxt_result = crypto_context.EvalAdd(ctxt_sum, bias)
```


Work in Progress: Upcoming Materials

 Coming Soon...

The following sections are currently under development and will be added to the training materials shortly:

Advanced BGV Challenge

A more complex and engaging programming task utilizing **homomorphic vector rotations** (EvalRotate) and SIMD packing to compute dot products and advanced metrics under the BGV scheme.

Introduction to the CKKS Scheme

A comprehensive section on **Approximate Real-Number Arithmetic**, detailing how to encode and homomorphically evaluate floating-point numbers (e.g., for AI and Machine Learning models).